Enterprise Doc AI

A Local-First Retrieval-Augmented Generation System for
Private Document Question Answering

Shahab Azmi
asaadkhalil1@gmail.com

May 2026

Abstract

Large language models (LLMs) provide fluent natural-language responses, yet they suffer from hallucination, knowledge cutoffs, and privacy concerns when applied to private documents. Cloud-hosted retrieval-augmented generation (RAG) services partially address hallucination but still require uploading sensitive content to third-party servers. We present *Enterprise Doc AI*, a local-first RAG system that runs entirely on commodity laptop hardware (Apple M2, 8 GB RAM) without any cloud or paid-API dependencies. The system combines an embedded Milvus Lite vector store, a 33 M-parameter bi-encoder for dense retrieval (`BAAI/bge-small-en-v1.5`), an Okapi BM25 lexical index, a hybrid reranker with domain-aware bonuses, and a 3-billion-parameter quantised generator (`llama3.2:3b` served by Ollama). A seven-class query classifier routes each request to a generative branch with type-specific prompt rules, an extractive branch that returns a verbatim chunk, or a metadata-only provenance branch – bypassing LLM inference entirely when the answer is purely a lookup. We describe the ingestion pipeline, hybrid retrieval, conversation memory, persistence design, and the engineering trade-offs required to make a small quantised model behave reliably. The deployed system indexes a 660-chunk multi-document corpus and answers grounded queries in 5–15 s end-to-end while never transmitting document content off-device.

1 Introduction

The last two years have seen large language models [1, 2] move from research curiosities into everyday productivity tools. For consumer use, hosted assistants such as ChatGPT and Claude are convenient. For *enterprise* or *personal* document question answering, however, three issues recur. First, cloud-hosted LLMs cannot be trusted with confidential material – contracts, medical records, internal HR documents, exam papers – whose loss would be material. Second, generic LLMs hallucinate when asked about facts outside their training data, and even retrieval-augmented chat services give the user no visibility into how the answer was constructed. Third, paid APIs are metered: every page indexed and every question asked has a unit cost that scales with usage, making them unsuitable for classroom or single-user research workflows.

Retrieval-augmented generation (RAG) [3, 4] addresses hallucination by grounding the model's answer in retrieved passages from an authoritative corpus. The system described in this report – *Enterprise Doc AI* – pushes the RAG idea to its local-first limit: the entire pipeline (parser, embedder, vector index, BM25 index, generator, and conversation store) runs on a single Apple M2 laptop with 8 GB of unified memory, no GPU, and no internet connection at query time. The 4 GB of resident memory available after the operating system is sufficient because every component is deliberately small: a 33 M-parameter bi-encoder, a Q4-quantised 3 B-parameter generator, an embedded vector store, and a SQLite relational backbone for documents and conversations.

The contributions of this work are:

- A complete, runnable **local-first RAG pipeline** that integrates dense retrieval (BGE [5]), Okapi BM25 [6], and a small quantised generator served by Ollama, with no cloud or paid-API dependency.
- A **seven-class query classifier** (EXTRACTIVE, PROVENANCE, NUMERICAL, LIST, REASONING, SUMMARY, FACTUAL) that routes each query to a different prompt template, retrieval depth, and token budget. Two of the seven classes *bypass* LLM inference entirely.

- A **hybrid reranker** combining normalised BM25 scores, normalised vector ranks, and a small set of domain-aware bonuses (e.g. extra weight for parenthetical acronym matches and for chunks containing email-shaped tokens), tuned to be useful on the small document corpora typical of personal use.
- A **table-aware chunker** that detects high-density numeric line patterns and preserves their line breaks instead of collapsing whitespace, which is essential for question answering over financial reports such as the 633-chunk Tesla 10-K used in our test corpus.
- A **persistence design** that stores documents and conversations in a single SQLite database (with WAL mode for concurrency), and rebuilds the embedded vector store from the canonical SQLite chunks on startup if the Milvus database file is missing.

The remainder of this report is organised as follows. Section 2 discusses prior work. Section 3 presents the system architecture. Section 4 details the implementation of each component. Section 5 reports on a qualitative evaluation on a heterogeneous four-document corpus. Section 6 discusses limitations and future work, and Section 7 concludes.

2 Related Work

Retrieval-augmented generation. Lewis et al. [3] introduced the modern RAG formulation, jointly training a retriever and generator on knowledge-intensive QA tasks. Subsequent surveys [4] catalogue dozens of variations – pre-, post-, and iterative-retrieval; query rewriting; self-reflection (Self-RAG [7]). Our system uses the simplest pre-retrieval variant but augments it with explicit query-type routing, which is rarely discussed in academic work but which we found essential for reliable behaviour with a small generator.

Sparse and dense retrieval. Okapi BM25 [6] remains the canonical sparse retrieval baseline – term frequency and inverse document frequency with length normalisation. Dense retrieval, popularised by DPR [8] and Sentence-BERT [9], embeds queries and documents into a shared vector space. The BGE family [5] we use is one of the strongest small open-source bi-encoders on the MTEB benchmark and runs at interactive speed on CPU. Hybrid sparse–dense retrieval is now standard practice; our weighted-rank-fusion variant is closest to the simple convex combination used in many production systems but adds domain bonuses that are tuned for personal-document (rather than web) corpora.

Vector databases and ANN. Milvus [10] is a purpose-built vector data management system. The *Milvus Lite* variant we use is an embedded, file-backed build that requires no separate server – ideal for single-machine deployment. Internally it uses HNSW [11] graphs for approximate nearest-neighbour search; we configure it with L2 distance and a similarity threshold of 1.2, above which retrieved chunks are discarded as too weak.

Local LLM serving. `llama.cpp` and Ollama [12] have made it practical to run quantised LLaMA-family [2] models on consumer hardware. We use Ollama as a managed local server for `llama3.2:3b` (Q4_K_M) – a 3-billion-parameter model quantised to four-bit weights, occupying ≈2 GB of disk and roughly the same in RAM during inference.

3 System Architecture

Enterprise Doc AI follows a three-tier design: a Streamlit single-page web client, a FastAPI service layer with seven cooperating modules, and a dual storage layer (SQLite for relational state plus Milvus Lite for vectors). Figure 1 sketches the data flow.

Local-first principles. Three rules guided the architecture: (i) the system must function fully offline once the LLM and embedding models are pulled; (ii) document content must never leave the device; (iii)

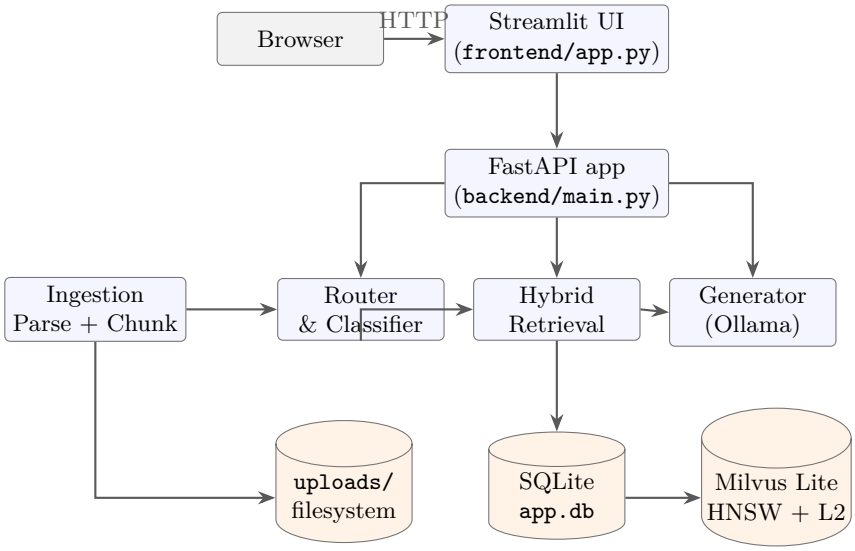

Figure 1. High-level architecture. Solid arrows indicate data flow during `/upload` and `/query` requests. The vector store is rebuilt from canonical SQLite chunks on startup if its file is missing, which makes the SQLite database the single source of truth.

state must survive process restarts without manual intervention. These rules motivated the choice of embedded vector store, the SQLite single-source-of-truth design, the SHA-256 deduplication on upload, and the auto-rebuild on startup.

Process model. The FastAPI `/query` endpoint is intentionally synchronous because both Milvus Lite and Ollama calls are blocking. Making the endpoint `async` would not parallelise these calls and would risk stalling the FastAPI event loop while the generator runs. A single `threading.Lock` serialises uploads and document deletions so concurrent calls cannot corrupt the vector store.

4 Implementation

This section walks through each pipeline stage. Throughout, file paths are given relative to the project root.

4.1 Document Ingestion and Chunking

The parser at `backend/services/parser.py` dispatches by file extension. PDFs are read page-by-page with PyMuPDF; one element per page is emitted, preserving the page number for later citation. DOCX files use `python-docx` and emit one element per paragraph, plus one element per table row joined by `" "|` – this preserves cell structure for retrieval. Plain text becomes a single element. Images go through `unstructured.partition.auto`, which transparently runs OCR.

Elements are then grouped by page and passed to a `RecursiveCharacterTextSplitter` with `chunk_size` = 500 and `chunk_overlap` = 100 characters. The splitter prefers paragraph boundaries first, falling back to single newlines, then to sentence boundaries, and finally to word boundaries. Each chunk inherits the source page's metadata: file name, page number, document UUID, and a heuristic *section* label.

Section detection. We scan the first six non-trivial lines of each page text block and classify a heading as the first match of: an all-uppercase line of length ≥ 4; a numbered heading (e.g. `1.1.`, `Chapter 3`, `II.`); or a short title-case line with no inline punctuation. The detected section becomes part of the citation displayed to the user.

Table-aware normalisation. Naive whitespace collapsing destroys tables: rows and columns merge into a single line. We detect this case heuristically – if at least three lines exist and more than 40 % of them are short (under 80 chars) and contain a digit, we keep the line breaks and only collapse intra-line whitespace:

```
def _normalize_chunk_text(text):
    lines = [l for l in text.split("\n") if l.strip()]
    numeric_short = sum(1 for l in lines
                        if len(l.strip()) < 80 and any(c.isdigit() for c in l))
    if len(lines) >= 3 and numeric_short / len(lines) > 0.4:
        return "\n".join(" ".join(l.split()) for l in lines)   # table mode
    return " ".join(text.split())                               # prose mode
```

This heuristic was added after observing that *Tesla, Inc. Form 10-K* pages – which contain many financial tables – were producing essentially unreadable single-line chunks before normalisation, severely hurting both BM25 and vector retrieval scores.

Embedding and indexing. Chunks are embedded with `BAAI/bge-small-en-v1.5` (384 dims, ≈ 33 M parameters) and inserted into Milvus Lite, which writes to a single file `milvus_demo.db`. The same chunks are mirrored into a SQLite `chunks` table whose metadata is JSON-serialised; on startup we check whether the Milvus file exists and, if not, we rebuild it from SQLite. A SHA-256 hash of the upload short-circuits re-ingestion of identical files.

4.2 Hybrid Retrieval

Given a query, retrieval (`backend/services/retrieval.py`) proceeds in four steps.
(1) Stop-word removal and alias expansion. A 50-word stop-list covers articles, common verbs, and document-meta words such as `pdf`, `document`, `paper`, `report`. A small alias map then expands `professor` to {`professor`, `prof`, `instructor`, `lecturer`, `teacher`}, `email` to {`email`, `mail`, `contact`}, and so on – a manual remedy for the well-known limitation of small bi-encoders on rare role nouns.
(2) BM25 candidate set. `rank-bm25`'s `BM25Okapi` is built over the tokens of all loaded chunks. The top $\max(3k, 12)$ chunks by BM25 score form the lexical candidate pool.
(3) Vector candidate set. A similarity search retrieves up to $\max(4k, 16)$ chunks from Milvus, after which any candidate with L2 distance > 1.2 is discarded. The threshold is tuned conservatively – a weak vector match contributes nothing useful and risks displacing a strong BM25 match during fusion.
(4) Rerank. Both pools are unioned and reranked by the convex combination

$$\text{score}(d) \;=\; w_b \cdot \widehat{b}(d) \;+\; (1 - w_b) \cdot \widehat{r}(d) \;+\; \beta(q, d),$$

where $\widehat{b}$ is the min–max-normalised BM25 score, $\widehat{r}$ is the normalised inverse vector rank, and β is a domain-aware bonus. The default weight $w_b = 0.5$ is increased to 0.7 for numerical queries (where exact term match matters more than semantic similarity). Bonuses are small (+0.10 to +0.18) and target specific cases: page 1 chunks for instructor queries; chunks containing `@` or `email` for contact queries; parenthetical occurrences such as `"natural language processing (NLP)"` for short-acronym queries; and chunks with `course description`, `course outcomes`, or `weekly schedule` for summary queries.

4.3 Query Classification

Before retrieval runs, the user's query is classified into one of seven types (Table 1). Classification is rule-based: the lower-cased query is checked against frozen sets of trigger phrases in priority order. The class then yields a `QueryConfig` dataclass that tells downstream stages how many chunks to retrieve, how to weight BM25 vs. dense scores, and how many tokens the LLM may emit.

The two "no LLM" rows are deliberate: an EXTRACTIVE query simply returns the highest- ranked chunk verbatim with a citation block; a PROVENANCE query returns only `Source/Page/Section` lines drawn from chunk metadata. Skipping the generator removes 5–10 s of latency and eliminates the small but non-zero risk of a 3 B model paraphrasing a quote that the user explicitly asked for verbatim.

4.4 Generation and Prompt Engineering

For the five generative classes, we issue a single Ollama call with `temperature` = 0 and `num_predict` set per Table 1. The prompt has a fixed five-rule core plus a type-specific addendum. The five rules

Table 1. Query types, retrieval parameters, and routing.

Type	k	w_b	n_{predict}	LLM?	Trigger examples
EXTRACTIVE	7	0.5	250	no	"exact paragraph", "verbatim", "quote from"
PROVENANCE	3	0.5	60	no	"which page", "what section", "where in"
NUMERICAL	5	0.7	60	yes	"how many", "percentage", "what year"
LIST	5	0.6	150	yes	"list all", "name all", "enumerate"
REASONING	5	0.4	120	yes	"why does", "what are the risks", "analyse"
SUMMARY	6	0.5	300	yes	"summarise", "overview of", "key points"
FACTUAL	3	0.5	120	yes	default

instruct the model to: use only the retrieved context, reply "Not found in document." when the context is silent, stop after one answer, prefer exact wording, and respect a 3–4-sentence ceiling unless explicitly asked otherwise. Type addenda add behaviour we found the 3 B model would otherwise violate – e.g. for REASONING, an explicit list of forbidden hedge phrases ("could lead to", "analysts believe", "market trends suggest") that the model would gladly invent if not blocked.

The retrieved context is concatenated with a 3000-character cap. We empirically found that filling the model's full ≈ 4 k token context window with chunks degraded answer quality on the 3 B model – a longer context was a worse context, not a better one, presumably because the model's effective attention budget is small. Each block is prefixed with a bracketed source/page/section header so the generator can ground citations.

4.5 Conversation Memory

Conversations and per-turn messages live in two SQLite tables linked by foreign key with `ON DELETE CASCADE`. `recent_turns(n=6)` returns the last six rows, which the generator formats as alternating `[User]`/`[AI]` lines under a header that explicitly warns the model not to copy the prior style. We cap turns at three (so six messages) because extra history did not measurably improve coherence on our test queries but did inflate prompt size noticeably. Conversations persist across server restarts and are listed in the sidebar.

5 Evaluation

We evaluate qualitatively on a four-document corpus exercised through the production UI. The documents and their chunk counts are given in Table 2.

Table 2. Test corpus indexed during the demonstration session.

Document	Format	Chunks
Tesla, Inc. Form 10-K (FY2024)	PDF	633
NLP Course Syllabus (HSE)	PDF	16
AI Homework 1	PDF	7
Resume (extended)	DOCX	4
Total		**660**

Hardware. Apple M2 with 8 GB of unified memory; no discrete GPU. Ollama uses Apple's Metal backend; the embedding model runs on the same backend through `sentence-transformers`.

Latencies (informal). Document upload and indexing of the 633-chunk financial PDF takes ≈ 22 s end-to-end (parsing, embedding, vector insertion, SQLite mirroring). Subsequent queries take 5–8 s for short factual answers, climbing to 12–18 s for summaries with longer `num_predict`. The first query after a cold start additionally pays a one-time ≈ 3 s model-load tax inside Ollama.

Walk-through. Figure 2 shows the UI in its empty state. Indexed documents are visible from the paperclip badge (`"4 indexed"`), and the conversation history is preserved across sessions in the left

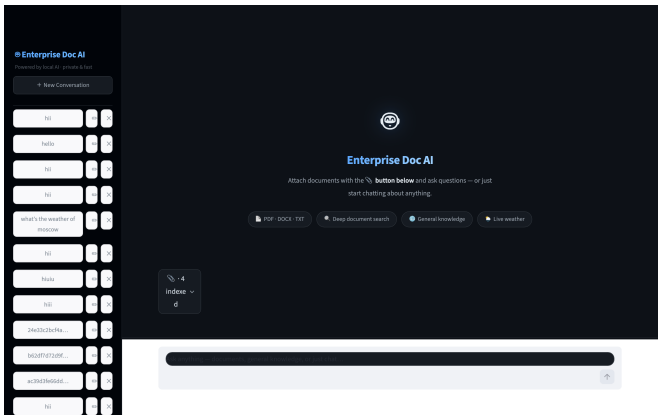

Figure 2. The Streamlit UI on a fresh load. The sidebar lists prior conversations recovered from SQLite; the centre panel shows the empty welcome state. Four documents are already indexed.

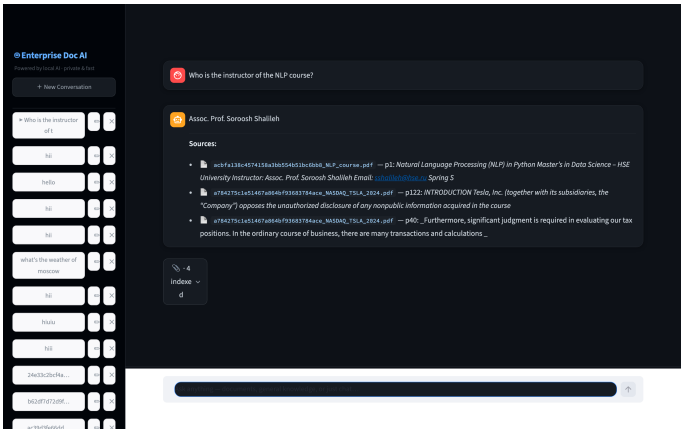

Figure 3. Grounded answer to a FACTUAL document query. The four-word answer is followed by three source citations rendered from chunk metadata (file, page, snippet).

sidebar.

Figure 3 shows the response to the question `"Who is the instructor of the NLP course?"`. The hybrid retriever surfaces three chunks – the top one from `NLP_course.pdf` page 1, plus two cross-document chunks for context. The generator returns the exact four-word answer "Assoc. Prof. Soroosh Shalileh", and the UI renders the source citations underneath. The +0.10 page-1 bonus and the +0.18 parenthetical-acronym bonus (acting on the literal string "Natural Language Processing (NLP)" in the chunk) both contribute to ranking the syllabus chunk above the much larger Tesla 10-K corpus despite the latter's higher chunk count.

Figure 4 shows a multi-turn session where a follow-up financial query is deliberately routed away from the document corpus. The router classified the question as a live-data query and the general-chat branch responded honestly that its training data ends in December 2023 – demonstrating that the system fails closed: when retrieval is not used, the LLM is allowed to admit ignorance rather than guess.

The upload pipeline (not pictured) supports multi-file selection of PDF, DOCX, TXT, PNG, and JPG content; SHA-256 deduplication on the backend means an identical file re-uploaded is detected and short-circuits within a few milliseconds, before any parsing or embedding work occurs.

6 Discussion and Future Work

Limitations. The most important limitation is the generator. `llama3.2:3b` at four-bit quantisation is roughly the smallest model that produces fluent English answers, and it occasionally drops a correct retrieved fact in favour of a near-paraphrase. We work around this with strict prompt rules (Section 4.4) and the LLM-bypass branches for extractive and provenance queries, but a step up to a 7 B–13 B model would noticeably raise the answer-quality ceiling. The cross-encoder rerank step common in production RAG [4] is absent: our hybrid lexical/dense fusion plus domain bonuses approximates it cheaply but is weaker than a true cross-encoder. The router that decides between document retrieval and general chat

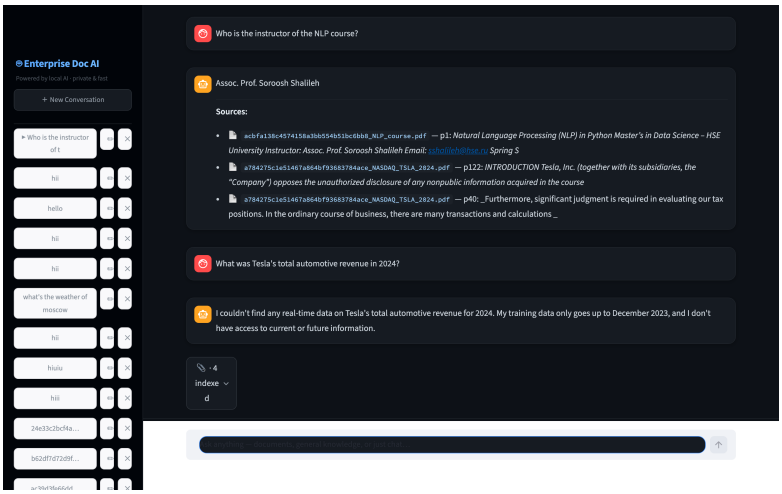

Figure 4. Multi-turn conversation. Turn 1 is grounded against documents; turn 2 is routed to the general-chat branch and the model openly states its knowledge limit instead of hallucinating a revenue figure.

is rule-based; queries about *indexed* corporate documents that look superficially like "ask the world" questions can be misrouted (visible in Figure 4).

Future work.

1. **Cross-encoder rerank.** Adding `BAAI/bge-reranker-base` as a second-stage reranker over the top 20 hybrid candidates would likely deliver the largest single retrieval-quality lift.
2. **Streaming responses.** Piping Ollama's token-stream API through FastAPI server-sent events into the Streamlit chat would change perceived latency from "5–15 s wait" to "immediate first token".
3. **PDF region linking.** Extending citation metadata with PyMuPDF text-rectangle coordinates would let the UI link a citation back to the source page region.
4. **Adaptive routing.** Replacing the rule-based document/chat router with a learned classifier would address the misrouting case in Figure 4.
5. **Per-conversation document scoping.** The retrieval layer already supports filtering by `doc_id`; a "search in" multi-select in the UI would let users restrict a conversation to a specific document.
6. **Multi-language embeddings.** Swapping the embedding model for the `bge-m3` multilingual variant would extend the system to non-English documents with no other code changes.

7 Conclusion

We have presented Enterprise Doc AI, a fully local retrieval-augmented generation system that runs on commodity laptop hardware. By combining a small bi-encoder, a classical BM25 index, a rule-based query classifier, and a quantised generator served by Ollama, the system delivers grounded document QA with source citations, conversation memory, and multi-document support without sending any content off-device. The engineering value of the work lies less in any single component than in their integration: type-aware routing, LLM-bypass for the cheapest question types, table-aware chunking, and a SQLite-as-source-of-truth design that lets the embedded vector store be safely dropped and rebuilt. The result is a system that a single user can run, understand, and extend on their own hardware – the local-first counterpart to hosted RAG products.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.

[5] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 641–649.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

[10] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021, pp. 2614–2627.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[12] Ollama Project, "Ollama: Get up and running with large language models locally," https://ollama.com, 2024, accessed: May 2026.